Greedy Algorithm

Tuesday, August 25, 2026 10:10 PM

① Dijkstra
② Kruskal / Prims
} greedy strategy/approach

③ Interval Scheduling problem

④ Huffman Encoding

Optimization problem
- min
- max

① DP S.C↑
② Greedy
③ Bruteforce T.C↑

Greedy Strategy -

⇒ Current context ↓ best possible decision 取る。

→ Two characteristics:

① Greedy choice property

② Optimal substructure

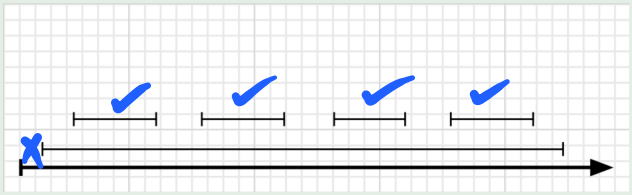
Strategy 1. *Earliest First*

The idea is to start using the resource as early as possible.

1. Sort the intervals by starting time, breaking ties arbitrarily.
2. Pick the first one, removing it from the list along with all the intervals that conflict with it.
3. Repeat Step 2, until the list is empty.

Optimal soln not guaranteed!

Example



$|A| = ???$.

$A = \{5\}$
$|A| = 1$

5

Strategy 2. *Shortest First*

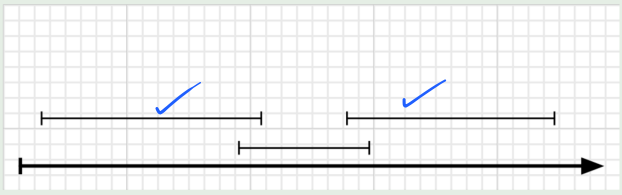
The *Earliest First* strategy failed perhaps because it missed the shorter intervals, which would accommodate more intervals.

1. Sort the intervals by length, breaking ties arbitrarily.
2. Pick the first one, removing it from the list along with all the intervals that conflict with it.
3. Repeat Step 2, until the list is empty.

Repeat Step 2, until the list is empty.

Optimal soln not guaranteed!

Example



$|A| = ???.$

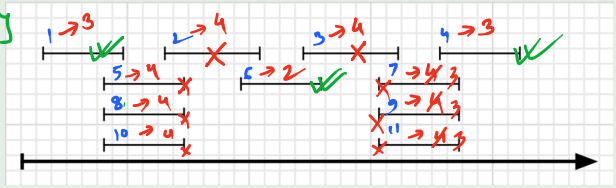
Strategy 3. *Least-conflict First*

The *Shortest First* strategy failed perhaps because the shorter ones had more conflicts, and ruled out too many intervals in the process.

1. Sort the intervals by the number of other intervals which conflict with it.
2. Pick the first one, removing it from the list along with all the intervals that conflict with it.
3. Repeat Step 2, until the list is empty.

Optimal soln not guaranteed!

Example



$|A| = ???.$

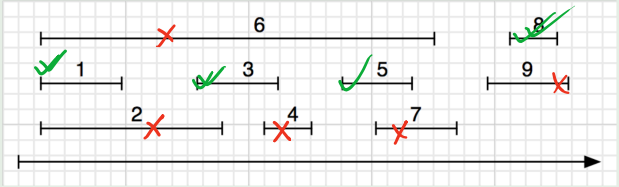
Strategy 4. *Finish First*

The idea is to free up the resource as early as possible.

1. Sort the intervals by the finishing time, breaking ties arbitrarily.
2. Pick the first one, removing it from the list along with all the intervals that conflict with it.
3. Repeat Step 2, until the list is empty.

Optimal soln is guaranteed!

Example



$|A| = ???.$

Extension: weighted interval scheduling problem

Definition (Weighted interval scheduling problem)

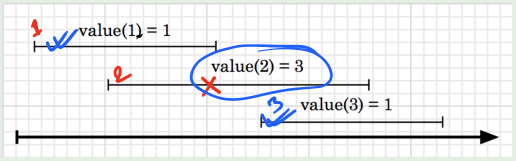
Given a set of schedules $I = \{I_i\}$, with associated weights $W = \{w_i\}$, find $A \subseteq I$ such that the members of A are non-conflicting *and* the total weight $\sum_{i \in A} w_i$ is maximized.

1 → 1

2 → 3

3 → 1

Example



$|A| = ???$, $\sum_{i \in A} w_i = ???$.

$A = \{1, 3\}$

$W_T = 1 + 1 = 2$

Greedy can not guarantee optimal soln

Huffman Encoding → bit compression technique / strategy

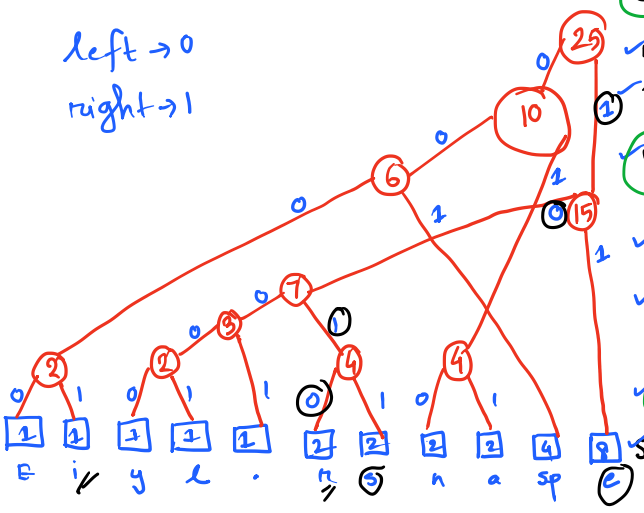
50 GB → (compression) 35 GB

- Consider the following short text:

Eerie, eyes, seen, near, lake.

left → 0

right → 1



Huffman Tree

Character	Count	Huffman Code	No. of bits	Total Bit
E	1	0000	4	1×4 = 4
e	8	11	2	8×2 = 16
n	2	1010	4	4×2 = 8
i	1	0001	4	1×4 = 4
y	1	10000	5	1×5 = 5
s	2	1011	4	2×4 = 8
n	2	010	3	2×3 = 6
a	2	011	3	2×3 = 6
l	1	10001	5	1×5 = 5
Space	4	001	3	4×3 = 12
.	1	1001	4	1×4 = 4
	25			78 bits

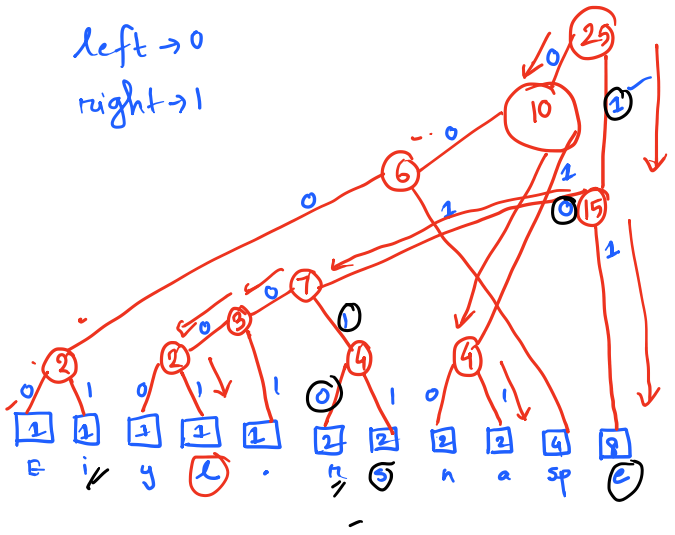
A → 65 (8 bit binary)

ASCII → fixed 8 bit encoding / 10 bit

Total bits = (25×8) = 200 bits

Bits saved = (200 − 78) = 122 bits

Huffman Decoding -



0 0 0 0 0 1 1 1 0 0 0 1 1 . . . . . .

E a l e